deux valeurs, comparer des données, accéder à une zone mémoire, modifier un registre, etc.

Pour cela, le processeur utilise notamment :

- des portes logiques, construites à partir de transistors ;
- des registres, qui permettent de stocker temporairement les valeurs utilisées pendant les calculs ;
- une unité arithmétique et logique, chargée d'effectuer les opérations de base.

Ayant toutes ces informations, vous décidez d'analyser un programme de calcul d'une suite de Fibonacci simplifiée*, développé en langage C.

Ce langage semble déjà beaucoup plus lisible que le binaire : il mélange des mots anglais, des symboles mathématiques et des structures de contrôle. Pourtant, le processeur, lui, ne comprend toujours que des instructions machine codées sous forme de 0 et de 1.

Étonnant : si le processeur ne manipule finalement que des signaux électriques, pourquoi le programme est-il écrit avec des mots comme *for*, *return*, *uint32_t* ou encore *add* ?

Vous comprenez rapidement qu'il serait impossible de coder efficacement directement en binaire. En revanche, vous découvrez qu'il existe un langage très proche du processeur : l'assembleur (ASM)

Ce langage permet d'écrire des instructions plus proches de ce qui sera réellement exécuté par le microcontrôleur. Sur la carte STM32 utilisée dans le projet, ces instructions appartiennent au jeu d'instructions Arm Thumb / Thumb-2.

Malheureusement pour vous, le code optimisé a été détérioré et vous voyez à peine l'instruction ASM apparaitre :

```c
#include <stdio.h>
#include "setup.h"

uint8_t fibonacci_asm(uint8_t iterations)
{
 uint32_t a = 1;
 uint32_t b = 2;
 uint32_t r = 0;

 for (uint8_t i = 0; i < iterations; i++)
 {
 uint32_t nouveau_a = 0;
 uint32_t nouveau_b = 0;

 /*
 * Le code assembleur ci-dessous a été détérioré.
 *
 * Votre objectif :
 * - calculer r = a + b ;
 * - conserver uniquement les 8 bits de poids faible ;
 * - préparer la prochaine itération avec a = b ;
 * - préparer la prochaine itération avec b = r.
 *
 * Instructions disponibles :
 * - ADDS : additionne deux registres
 * - UXTB : conserve uniquement les 8 bits de poids faible
 * - MOV : copie une valeur d'un registre vers un autre
 */

 __asm volatile (
 "____ %[resultat], %[valeur_a], %[valeur_b]\n"
 "____ %[resultat], %[resultat]\n"
 "____ %[nouveau_a], %[valeur_b]\n"
 "____ %[nouveau_b], %[resultat]\n"
 : [resultat] "=&r" (r),
 [nouveau_a] "=&r" (nouveau_a),
 [nouveau_b] "=&r" (nouveau_b)
 : [valeur_a] "r" (a),
 [valeur_b] "r" (b)
 : "cc"
 );
```